Ex.No. :02

Capturing and Analyzing the Packet Traces of HTTP

Aim

To capture and analyzes the request and resply packet traces of HTTP protocol using Wireshark

HTTP

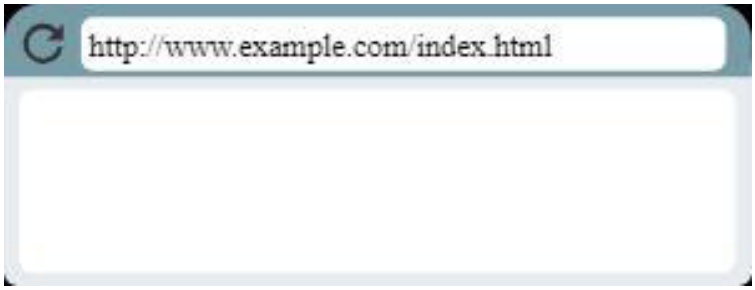
- HTTP stands for **HyperText Transfer Protocol**.It is a protocol used to access the data on the World Wide Web (www).
- The HTTP protocol can be used to transfer the data in the form of plain text, hypertext, audio, video, and so on. It is used to carry the data in the form of MIME-like format.
- HTTP is similar to the FTP as it also transfers the files from one host to another host. But, HTTP is simpler than FTP as HTTP uses only one connection, i.e., no control connection to transfer the files.
- HTTP is similar to SMTP as the data is transferred between client and server. The HTTP differs from the SMTP in the way the messages are sent from the client to the server and from server to the client. SMTP messages are stored and forwarded while HTTP messages are delivered immediately.

Working of HTTP

First of all, whenever to open any website, first open a web browser after that type the URL of that website (e.g., www.facebook.com). This URL is now sent to Domain Name Server (DNS). Then DNS first check records for this URL in their database, then DNS will return the IP address to the web browser corresponding to this URL. Now the browser is able to send requests to the actual server.

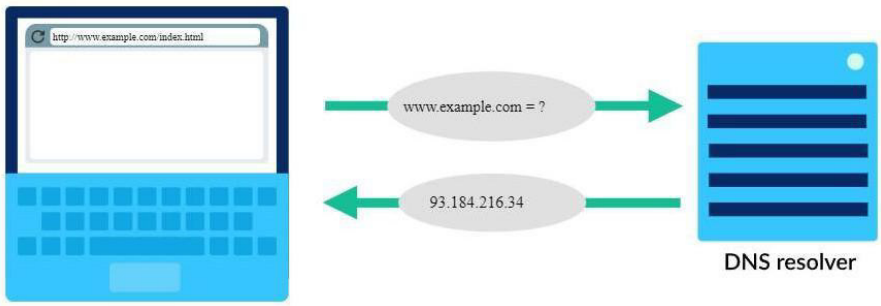
After the server sends data to the client, the connection will be closed. If we want something else from the server we should have to re-establish the connection between the client and the server.

Step 1: Direct browser to URL



Step 2: Browser looks up IP

The browser uses a DNS resolver to map the domain to an IP address:



Step 3: Browser sends HTTP request

Once the browser identifies the IP address of the computer hosting the requested URL, it sends an **HTTP request**.



An HTTP request can be as short as two lines of text:

GET /index.html HTTP/1.1 Host: www.example.com

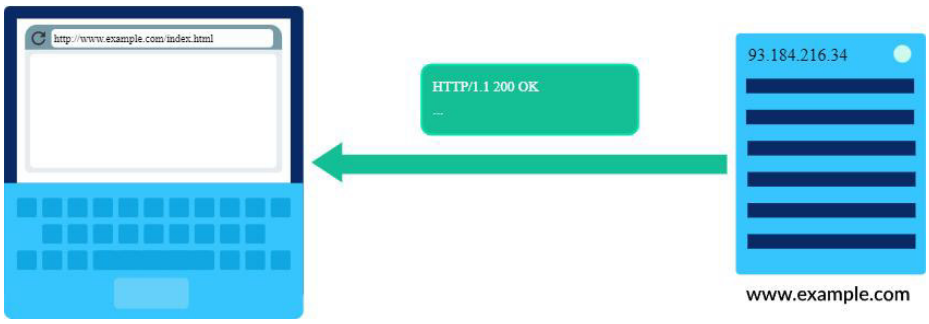
The first word is the HTTP verb: "GET". There are other verbs for other actions on the web, like submitting form data ("POST"). The next part specifies the path: "/index.html". The host computer stores the content of the entire website, so the browser needs to be specific about which page to load.

The final part of the first line specifies the protocol and the version of the protocol: "HTTP/1.1". The second line specifies the domain of the requested URL. That's helpful in case a host computer stores the content for multiple websites.

Step 4: Host sends back HTTP response

Once the host computer receives the HTTP request, it sends back a response with both the content and metadata about it.

The HTTP response starts similarly to the request:



The response begins with the protocol and version, "HTTP/1.1".

The next number is the very important **HTTP status code**, and in this case, it's 200. That code represents a successful retrieval of the document ("OK"). If the server failed to retrieve the document, the status codes provide more information, like if the failure was due to user error or server error. For example, the most well known status code is 404 ("File not found").

The next part of an HTTP response are the **headers**. They give the browser additional details and help the browser to render the content.

These two headers are common to most requests:

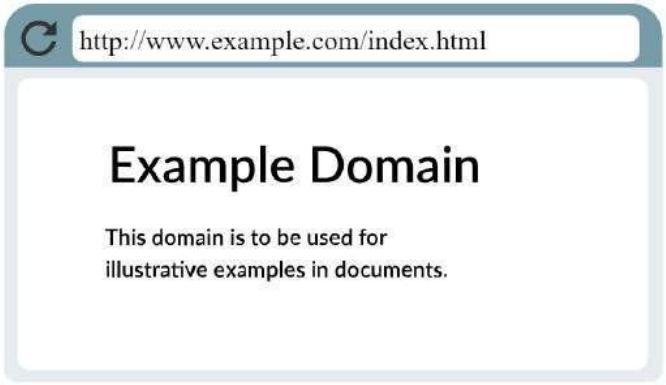
Content-Type: text/html; charset=UTF-8 Content-Length: 208

The content-type tells the browser what type of document it's sending back. A common type on the web is "text/html", as all webpages are HTML text files. Other types are possible, like images ("image/png"), videos ("video/mpeg"), script ("application/javascript") and anything else you can load in your browser.

The content-length gives the length of the document in bytes, which helps the browser know how long a file will take to download. Finally, the HTTP response writes out the actual document requested.

Step 5: The browser renders the response

The browser now has all the information it needs to render the document requested.



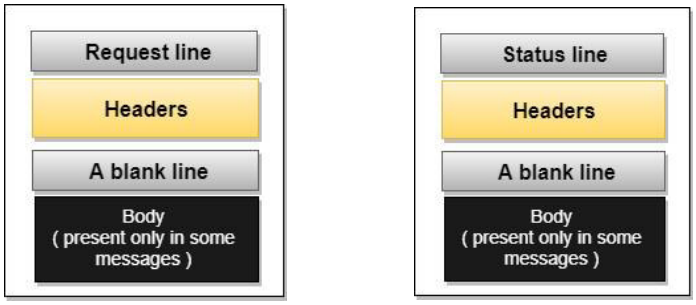
Features of HTTP:

- **Connectionless protocol:** HTTP is a connectionless protocol. HTTP client initiates a request and waits for a response from the server. When the server receives the request, the server processes the request and sends back the response to the HTTP client after which the client disconnects the connection. The connection between client and server exist only during the current request and response time only.
- **Media independent:** HTTP protocol is a media independent as data can be sent as long as both the client and server know how to handle the data content. It is required for both the client and server to specify the content type in MIME-type header.
- **Stateless:** HTTP is a stateless protocol as both the client and server know each other only during the current request. Due to this nature of the protocol, both the client and server do not retain the information between various requests of the web pages.

Messages

HTTP messages are of two types: request and response. Both the message types follow the same message format.

Request Message: The request message is sent by the client that consists of a request line, headers, and sometimes a body.



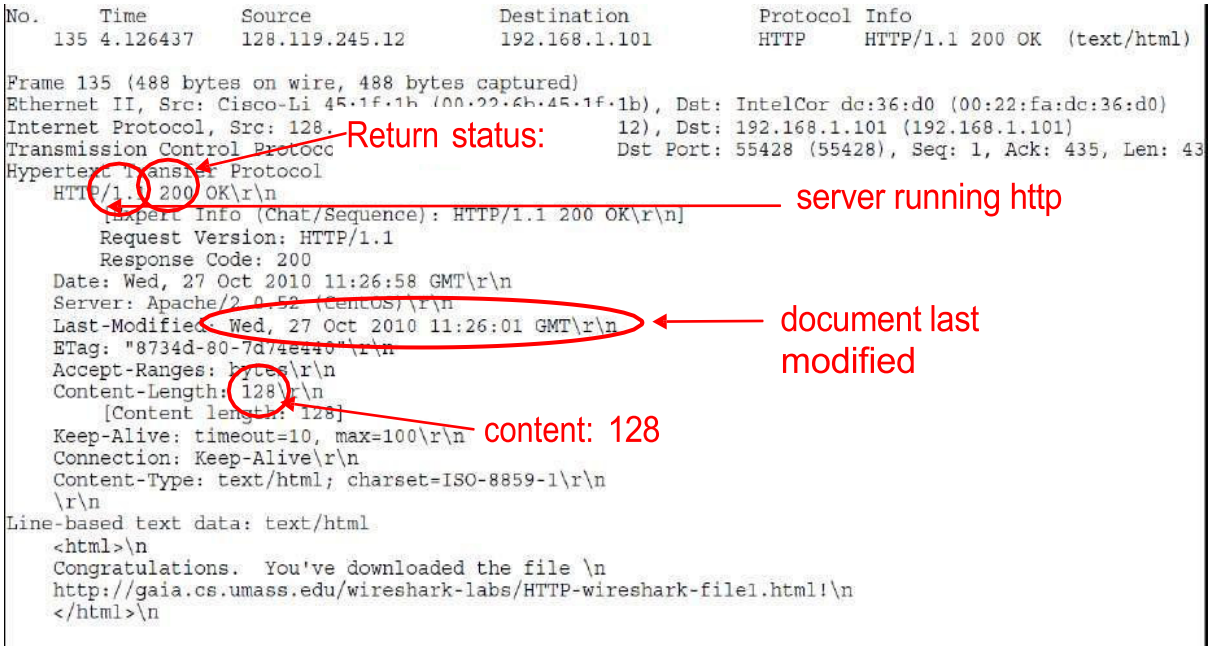
Response Message: The response message is sent by the server to the client that consists of a status line, headers, and sometimes a body.

1. Is your browser running HTTP version 1.0 or 1.1? What version of HTTP is the server running?
2. What languages (if any) does your browser indicate that it can accept to the server?
3. What is the IP address of your computer? Of the gaia.cs.umass.edu server?

```
No.     Time        Source                Destination           Protocol Info
    133 4.098946    192.168.1.101         128.119.245.12        HTTP     GET /wireshark-labs/HTTP-wire

Frame 133 (488 bytes on wire, 488 bytes captured)
Ethernet II, Src: IntelCor_dc:36:d0 (00:22:fa:dc:36:d0), Dst: Cisco-Li_45:1f:1b (00:22:6b:45:1f:1b)
Internet Protocol, Src: 192.168.1.101 (192.168.1.101), Dst: 128.119.245.12 (128.119.245.12)
    Version: 4
    Header length: 20 bytes
    Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00)
    Total Length: 474
    Identification: 0x036e (878)
    Flags: 0x02 (Don't Fragment)
    Fragment offset: 0
    Time to live: 128
    Protocol: TCP (0x06)
    Header checksum: 0xbe1e [correct]
    Source: 192.168.1.101 (192.168.1.101)
    Destination: 128.119.245.12 (128.119.245.12)
Transmission Control Protocol, Src Port: 55428 (55428), Dst Port: http (80), Seq: 1, Ack: 1, Len: 434
    Source port: 55428 (55428)
    Destination port: http (80)
    [Stream index: 27]
    Sequence number: 1    (relative sequence number)
    [Next sequence number: 435    (relative sequence number)]
    Acknowledgement number: 1    (relative ack number)
    Header length: 20 bytes
    Flags: 0x18 (PSH, ACK)
    Window size: 64240
    Checksum: 0xe737 [validation disabled]
    [SEQ/ACK analysis]
Hypertext Transfer Protocol
    GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1\r\n
    Host: gaia.cs.umass.edu\r\n
    User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.1; en-US; rv:1.9.2.11) Gecko/20101012 Firefox/3.
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-us,en;q=0.5\r\n
    Accept-Encoding: gzip,deflate\r\n
    Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7\r\n
    Keep-Alive: 115\r\n
    Connection: keep-alive\r\n
    \r\n
```

4. What is the status code returned from the server to your browser?
5. When was the HTML file that you are retrieving last modified at the server?
6. How many bytes of content are being returned to your browser?



Result:

Thus, capturing and analyzing HTTP packets using Wireshark has been completed successfully.